FIAP

RELATÓRIO TÉCNICO
ETAPA 3

Sistema A.D.P. — Análise, Decisão e Previsão
Colônia de Marte

Henrique Floriano Alcantara
Nicolas França
Joao Pedro Candido Souza
Katia Regina Bispo
Yago Souza Araujo

Ciência da Computação | 2026

1. Introdução

A Etapa 3 da missão Aurora propõe um sistema integrado para apoiar a operação de uma colônia em Marte. O desafio reúne organização de dados, decisões por condições lógicas, análise energética e uma previsão simples baseada em dados históricos. Desenvolvemos o Sistema A.D.P. (Análise, Decisão e Previsão) em Python para reunir essas tarefas em um único programa de terminal.

O sistema recebe informações sobre energia disponível, geração, consumo e velocidade do vento. A partir desses valores, apresenta alertas ou sugestões e estima a geração eólica com uma regressão linear. É um protótipo didático: não recebe dados de sensores reais nem controla equipamentos físicos.

2. Objetivo e escopo

O objetivo foi construir um programa funcional que organize informações da colônia e transforme entradas numéricas em respostas fáceis de interpretar. A proposta também permite praticar funções, estruturas condicionais, laços de repetição, tratamento de entradas inválidas e um modelo matemático simples.

A implementação foi mantida próxima ao que o enunciado solicita. O foco está no funcionamento da lógica e na clareza da organização, não em simular uma infraestrutura energética real com precisão física.

3. Organização dos dados

Para a previsão, o programa guarda dois históricos em listas: vento_historico = [8, 10, 12] e energia_historico = [20, 25, 30]. Cada posição de uma lista corresponde à mesma posição da outra: vento de 8 km/h está associado a 20 unidades de energia, por exemplo.

Os subsistemas são representados por um dicionário chamado sistemas. A chave energetico contém outro dicionário, com solar e eolico marcados como verdadeiros. As chaves suporte_vida, iluminacao e lazer completam a representação. Essa estrutura permite reunir dados relacionados e representar uma hierarquia simples.

```
sistemas = {
    "energetico": {"solar": True, "eolico": True},
    "suporte_vida": True,
    "iluminacao": False,
    "lazer": False
}
```

No código atual, o dicionário representa estados definidos previamente. A opção de menu “Mostrar sistemas” apenas exibe esses estados; não altera os valores.

4. Regras de decisão

4.1 Comparação entre geração e consumo

A função analisar_energia(gerac, cons) compara os dois valores. Quando o consumo supera a geração, o programa mostra um alerta. Se a geração for maior, sugere armazenar o excedente. Quando os valores são iguais, informa que existe equilíbrio.

4.2 Verificação de energia disponível

A função decidir(energ, cons) aplica condições em sequência. Se a energia disponível for menor que 50 e o consumo for maior que 70, recomenda entrar em modo de economia e desligar o que não for essencial. Se apenas a energia for menor que 50, recomenda reduzir o consumo. Nos demais casos, apresenta a mensagem de operação normal.

A regra combinada usa o operador lógico and. É importante distinguir a mensagem da ação: o código recomenda desligar sistemas não essenciais, mas não executa esse desligamento automaticamente.

5. Previsão de energia eólica

A função prever_energia(vento) utiliza numpy.polyfit com grau 1 para ajustar uma reta aos três pares históricos. Com esses dados, a relação encontrada é energia estimada = 2,5 × vento. Para um vento de 11 km/h, a saída é 27,50 unidades.

```
m, b = np.polyfit(vento_historico, energia_historico, 1)
energia_nova = m * vento + b
```

Essa estimativa demonstra a aplicação de regressão linear simples estudada na etapa. Como a amostra contém somente três observações e foi definida para o exercício, o resultado não deve ser interpretado como uma previsão validada da produção eólica real em Marte.

6. Funcionamento do programa

O sistema inicia com um menu em repetição contínua. Na opção 1, o usuário cadastra energia disponível, geração, consumo e velocidade do vento. As opções 2, 3 e 4 executam, respectivamente, a análise energética, a decisão por regras e a previsão eólica. A opção 5 exibe o dicionário dos sistemas e a opção 0 encerra o programa.

O código organiza as três operações principais em funções e usa try/except ValueError para lidar com entradas que não podem ser convertidas em números. Também impede o uso das opções de análise antes do cadastro. A versão atual não rejeita explicitamente valores negativos, e um novo cadastro substitui os dados anteriores em vez de armazenar um histórico de execuções.

7. Exemplos de entrada e saída

Os cenários abaixo foram derivados das condições do código e dos dados apresentados no relatório de referência. Eles demonstram as saídas esperadas; não representam uma bateria independente de testes executada pela equipe.

Situação	Entrada	Resultado esperado
Consumo acima da geração	geração = 40; consumo = 70	ALERTA: consumo maior que geração!
Excedente energético	geração = 80; consumo = 30	SUGESTÃO: armazenar energia excedente!
Energia crítica e consumo alto	energia = 40; consumo = 80	Recomendação de modo de economia
Previsão eólica	vento = 11 km/h	27,50 unidades de energia

8. Contribuição para a gestão da colônia

O programa permite identificar rapidamente uma situação em que o consumo supera a geração e sugere uma resposta quando a energia disponível está baixa. Também mostra como dados históricos podem servir de base para uma estimativa. Na prática, essas três funções ajudam a demonstrar a diferença entre observar dados, aplicar regras e produzir previsões.

Por enquanto, o protótipo não considera reservas acumuladas no cálculo, prioridades automáticas de desligamento, variações climáticas adicionais ou desempenho real dos equipamentos. Esses pontos seriam possibilidades de evolução, mas não fazem parte da implementação apresentada.

9. Considerações finais

O Sistema A.D.P. reúne em um único programa os conceitos centrais pedidos na atividade: listas e dicionários, organização hierárquica, funções, estruturas condicionais, análise de geração e consumo e regressão linear simples. O menu facilita a demonstração de cada parte sem exigir que o usuário altere o código.

O principal resultado é a integração dos conteúdos em um exercício concreto. A implementação também deixa claro o que ainda poderia ser melhorado, como validar limites de entrada, registrar medições e transformar recomendações em alterações simuladas de estado.

10. Referências e materiais consultados

FIAP. A Aurora Estabelece os Primeiros Sistemas da Colônia. Capítulo 1, 2026. Enunciado e critérios de avaliação.

FIAP. A Estrutura de Armazenamento Organizando os Dados que Sustentam a Colônia. Capítulo 2, 2026. Lógica booleana.

FIAP. A Lógica Estruturada que Dá Forma às Primeiras Soluções da Base. Capítulo 3, 2026. Funções e programação estruturada.

FIAP. As Estruturas Avançadas que Rastreiam Recursos e Informações da Colônia. Capítulo 4, 2026. Estruturas de dados.

FIAP. Os Vetores e Matrizes que Simulam o Comportamento Inicial da Infraestrutura. Capítulo 5, 2026. Vetores e NumPy.

FIAP. O Sistema Operacional que Orquestra os Processos da Colônia. Capítulo 6, 2026. Contexto computacional.

FIAP. A Regressão Inicial que Permite Prever Variáveis da Nova Base. Capítulo 7, 2026. Regressão linear simples.

FIAP. As Fontes Renováveis que Alimentam os Primeiros Sistemas da Colônia. Capítulo 8, 2026. Energia solar e eólica.

FRANÇA, Nicolas. Relatório de Atividade de Extensão — FIAP (Parte 2 — Ajustado). 2026. Material complementar da equipe.

11. Repositório público

Repositório público do projeto:
https://github.com/YagoSDev/Sistema_ADP_Colonia_Marte